

```

<!--
{
  "availability" : [
    "iOS: 8.0.0 -",
    "iPadOS: 8.0.0 -",
    "macCatalyst: 13.0.0 -",
    "macOS: 10.10.0 -",
    "tvOS: 9.0.0 -",
    "visionOS: 1.0.0 -",
    "watchOS: 2.0.0 -"
  ],
  "documentType" : "symbol",
  "framework" : "Swift",
  "identifier" : "/documentation/Swift/CustomDebugStringConvertible",
  "metadataVersion" : "0.1.0",
  "role" : "Protocol",
  "symbol" : {
    "kind" : "Protocol",
    "modules" : [
      "Swift"
    ]
  },
  "preciseIdentifier" : "s:s28CustomDebugStringConvertibleP"
},
{
  "title" : "CustomDebugStringConvertible"
}
-->

```

CustomDebugStringConvertible

A type with a customized textual representation suitable for debugging purposes.

```

...

```

```

protocol CustomDebugStringConvertible
{
}

```

Overview

Swift provides a default debugging textual representation for any type. That default representation is used by the `String(reflecting:)` initializer and the `debugPrint(_:)` function for types that don't provide their own. To customize that representation, make your type conform to the `CustomDebugStringConvertible` protocol.

Because the `String(reflecting:)` initializer works for instances of *any* type, returning an instance's `debugDescription` if the value passed conforms to `CustomDebugStringConvertible`, accessing a type's `debugDescription` property directly or using `CustomDebugStringConvertible` as a generic constraint is discouraged.

> Note: Calling the `dump(_:_:_:_:)` function and printing in the debugger uses both `String(reflecting:)` and `Mirror(reflecting:)` to collect information about an instance. If you implement `CustomDebugStringConvertible` conformance for your custom type, you may want to consider providing a custom mirror by implementing `CustomReflectable` conformance, as well.

Conforming to the CustomDebugStringConvertible Protocol

Add `CustomDebugStringConvertible` conformance to your custom types by defining a `debugDescription` property.

For example, this custom `Point` struct uses the default representation supplied by the standard library:

```
```
struct Point {
 let x: Int, y: Int
}

let p = Point(x: 21, y: 30)
print(String(reflecting: p))
// Prints "Point(x: 21, y: 30)"
```
```

After adding `CustomDebugStringConvertible` conformance by implementing the `debugDescription` property, `Point` provides its own custom debugging representation.

```
```
extension Point: CustomDebugStringConvertible {
 var debugDescription: String {
 return "(\(x), \(y))"
 }
}

print(String(reflecting: p))
// Prints "(21, 30)"
```
```

Relationships

Conforming Types

```
[`CValistPointer`](/documentation/Swift/CValistPointer)
[`PartialKeyPath`](/documentation/Swift/PartialKeyPath)
[`UnsafeMutableBufferPointer`](/documentation/Swift/UnsafeMutableBufferPointer)
[`String`](/documentation/Swift/String)
[`DecodingError`](/documentation/Swift/DecodingError)
[`UnsafeRawBufferPointer`](/documentation/Swift/UnsafeRawBufferPointer)
[`StaticBigInt`](/documentation/Swift/StaticBigInt)
[`StaticString`](/documentation/Swift/StaticString)
[`Dictionary`](/documentation/Swift/Dictionary)
[`ContiguousArray`](/documentation/Swift/ContiguousArray)
[`Keys-swift.struct`](/documentation/Swift/Dictionary/Keys-swift.struct)
[`SIMD4`](/documentation/Swift/SIMD4)
[`AutoreleasingUnsafeMutablePointer`](/documentation/Swift/AutoreleasingUnsafeMutablePointer)
[`WordPair`](/documentation/Synchronization/WordPair)
[`Substring`](/documentation/Swift/Substring)
```

[`ObjectIdentifier`](/documentation/Swift/ObjectIdentifier)

[`Float80`](/documentation/Swift/Float80)

[`UnsafeBufferPointer`](/documentation/Swift/UnsafeBufferPointer)

[`UnsafeRawPointer`](/documentation/Swift/UnsafeRawPointer)

[`Set`](/documentation/Swift/Set)

[`UTF8View`](/documentation/Swift/String/UTF8View)

[`EncodingError`](/documentation/Swift/EncodingError)

[`ArraySlice`](/documentation/Swift/ArraySlice)

[`ReferenceWritableKeyPath`](/documentation/Swift/ReferenceWritableKeyPath)

[`Values-swift.struct`](/documentation/Swift/Dictionary/Values-swift.struct)

[`SIMD16`](/documentation/Swift/SIMD16)

[`UnsafeMutableRawBufferPointer`](/documentation/Swift/UnsafeMutableRawBufferPointer)

[`KeyPath`](/documentation/Swift/KeyPath)

[`UnsafeMutablePointer`](/documentation/Swift/UnsafeMutablePointer)

[`Index`](/documentation/Swift/String/Index)

[`Array`](/documentation/Swift/Array)

[`AnyHashable`](/documentation/Swift/AnyHashable)

[`ClosedRange`](/documentation/Swift/ClosedRange)

[`UTF16View`](/documentation/Swift/String/UTF16View)

[`Range`](/documentation/Swift/Range)

[`SIMD8`](/documentation/Swift/SIMD8)

[`Character`](/documentation/Swift/Character)

[`SIMD64`](/documentation/Swift/SIMD64)

[`UnsafePointer`](/documentation/Swift/UnsafePointer)

[`SIMD32`](/documentation/Swift/SIMD32)

[`Scalar`](/documentation/Swift/Unicode/Scalar)

[`SIMD3`](/documentation/Swift/SIMD3)

[`SIMD2`](/documentation/Swift/SIMD2)

[`Optional`](/documentation/Swift/Optional)

[`Float16`](/documentation/Swift/Float16)

[`OpaquePointer`](/documentation/Swift/OpaquePointer)

[`AnyKeyPath`](/documentation/Swift/AnyKeyPath)

[`Double`](/documentation/Swift/Double)

[`UnsafeMutableRawPointer`](/documentation/Swift/UnsafeMutableRawPointer)

[`Float`](/documentation/Swift/Float)

[`CollectionOfOne`](/documentation/Swift/CollectionOfOne)

[`WritableKeyPath`](/documentation/Swift/WritableKeyPath)

[`UnicodeScalarView`](/documentation/Swift/String/UnicodeScalarView)

[`KeyValuePairs`](/documentation/Swift/KeyValuePairs)

Inherited By

[`CodingKey`](/documentation/Swift/CodingKey)

Copyright © 2026 Apple Inc. All rights reserved. | [Terms of Use](https://www.apple.com/legal/internet-services/terms/site.html) | [Privacy Policy](https://www.apple.com/privacy/privacy-policy)